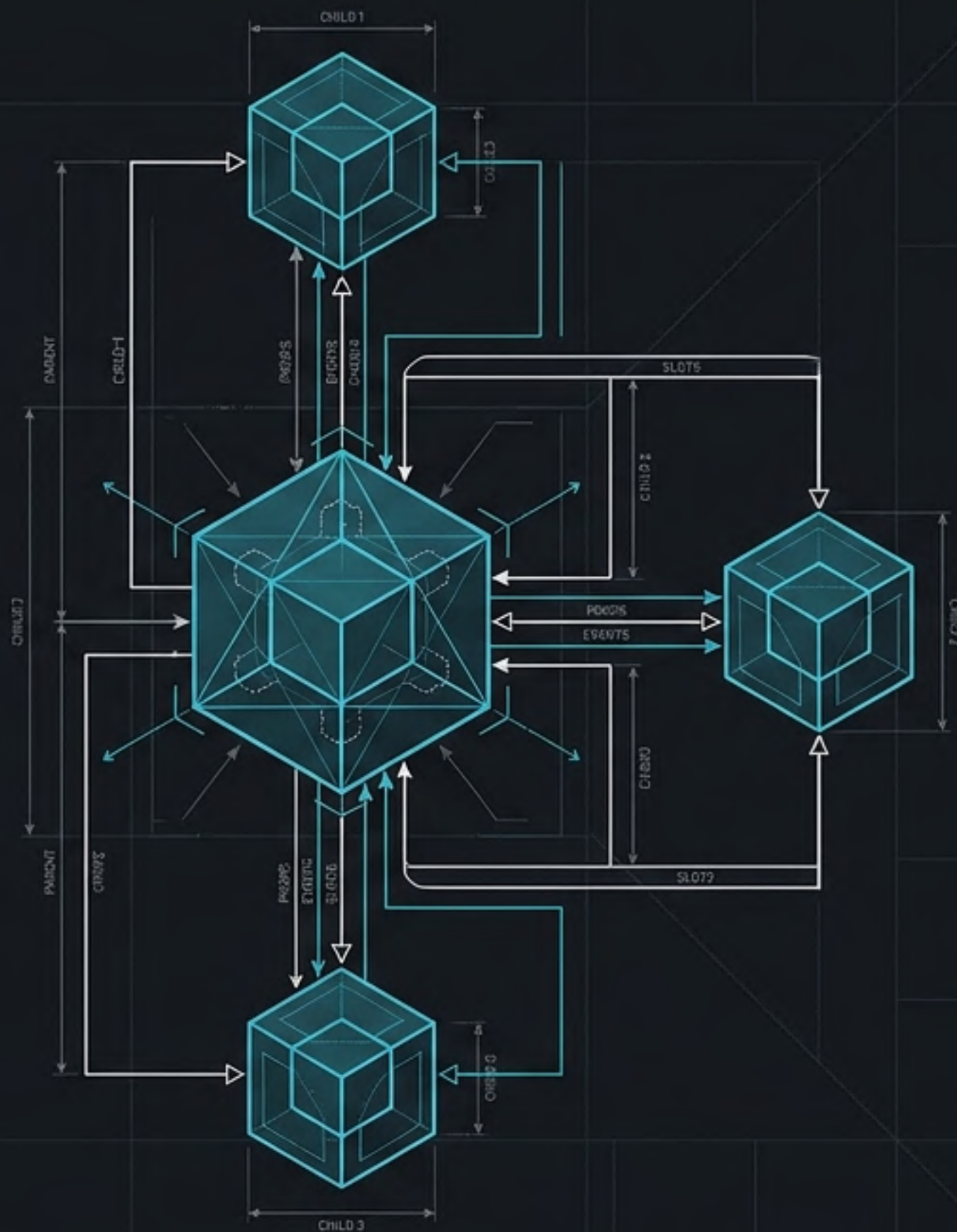


Componentes y ciclo de vida en Vue 3

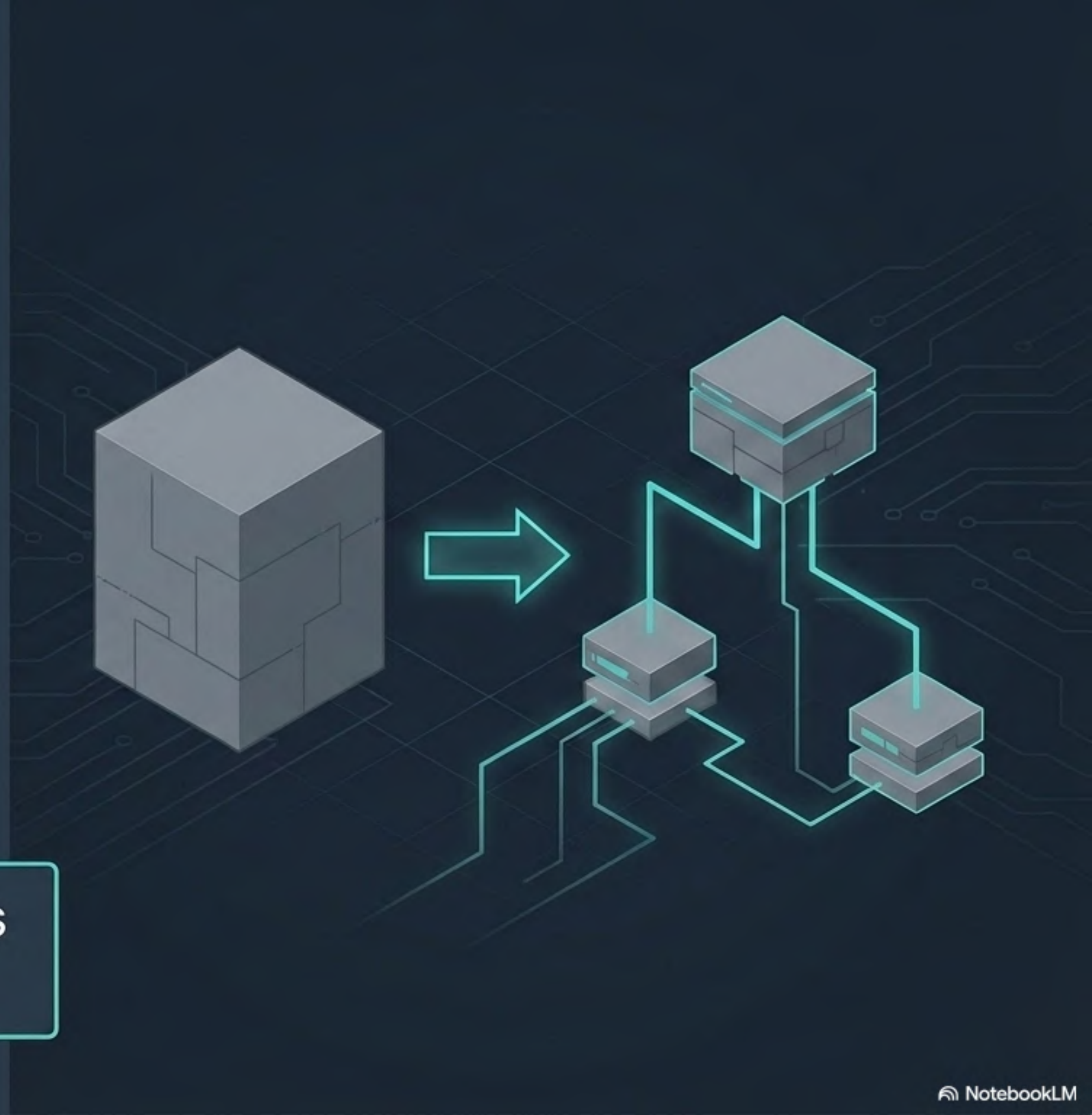
Comunicación, composición y etapas internas de una arquitectura escalable.



De una vista única a un ecosistema

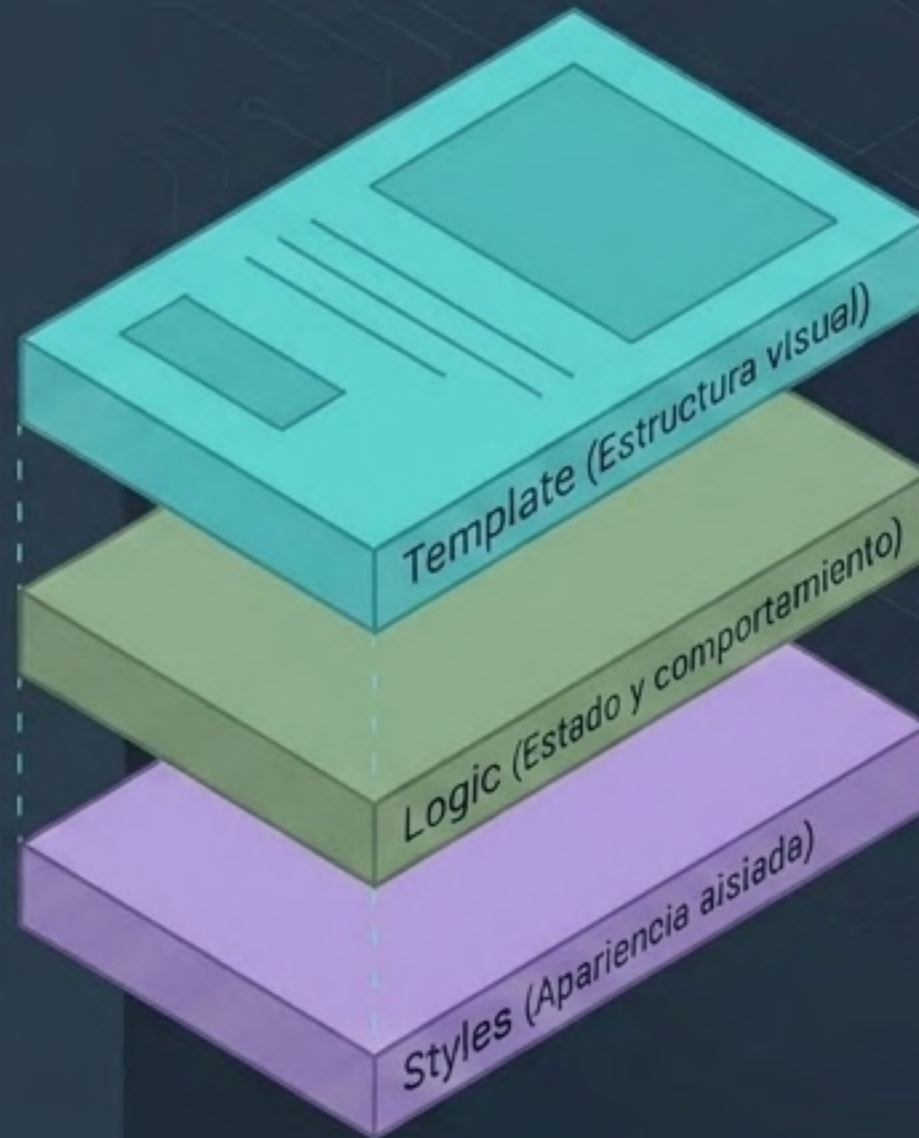
Antes construíamos la interfaz en un solo bloque. Ahora, dividimos la aplicación en piezas especializadas, independientes y conectadas. En esta lección no buscamos aprender nuevas directivas, sino organizar mejor lo que ya sabemos en componentes conectados.

Vue se escala mediante jerarquías estrictas de componentes.



Componentización: El bloque de construcción

Un componente Vue es una pieza de UI independiente que agrupa template, lógica y estilos dentro de un archivo .vue.



Single-File Component (SFC) Moderno

Template: estructura declarativa de la interfaz.

```
<template>
  <section class="card">
    <h2>{{ titulo }}</h2>
  </section>
</template>

<script setup>
  const titulo = 'Componente Vue'
</script>

<style scoped>
  .card { border: 1px solid #22c55e; }
</style>
```

Elimina el boilerplate.

Aísla el CSS solo para este componente.

Jerarquía: App → Padre

En `<script setup>`, los componentes importados quedan disponibles automáticamente en el `template`.

Se usan como etiquetas en PascalCase.



```
<script setup>
import Padre from './components/Padre.vue'
</script>

<template>
  <Padre />
</template>
```



Props: Comunicación Padre → Hijo

Props envían datos desde el padre hacia el hijo.

En el hijo son de solo lectura: se reciben y muestran, pero no se modifican directamente.

```
<script setup>
import Hijo from './Hijo.vue'
const mensajePadre = 'Hola desde el padre'
</script>
<template>
  <Hijo :mensaje="mensajePadre" />
</template>
```



Recibiendo Props con defineProps

Es una macro de compilación exclusiva de <script setup>. No necesita importarse y provee validación de tipos y requerimientos en tiempo de desarrollo.

- **defineProps** no se importa: es una macro de <script setup>.
- **type** valida el tipo esperado y **required** indica si la prop es obligatoria.

```
<script setup>
defineProps({
  mensaje: { type: String, required: true }
})
</script>
<template>
  <p>{{ mensaje }}</p>
</template>
```

Validación estricta de tipos

Emits: Comunicación Hijo → Padre

Eventos personalizados que el hijo dispara para notificar acciones al padre. El nombre emitido debe coincidir con el nombre escuchado por el padre.

```
<script setup>
const emit = defineEmits(['respuestaHijo'])

function responder() {
  emit('respuestaHijo', 'Hola padre, soy el hijo')
}
</script>

<template>
  <button @click="responder">Responder</button>
</template>
```



Escuchando el Emit en el Padre

```
<script setup>
import { ref } from 'vue'
import Hijo from './Hijo.vue'
```

```
const respuesta = ref('')
```

```
function recibirRespuesta(mensaje) {
  respuesta.value = mensaje
}
```

```
</script>
```

```
<template>
```

```
  <Hijo mensa @respuestaHijo="recibirRespuesta"
```

```
  "recibirRespue
```

```
  <p v-if="respuesta">{{ respuesta }}</p>
```

```
</template>
```

emit('respuestaHijo')

El padre escucha el evento personalizado y ejecuta una función para actualizar su propio estado reactivo.

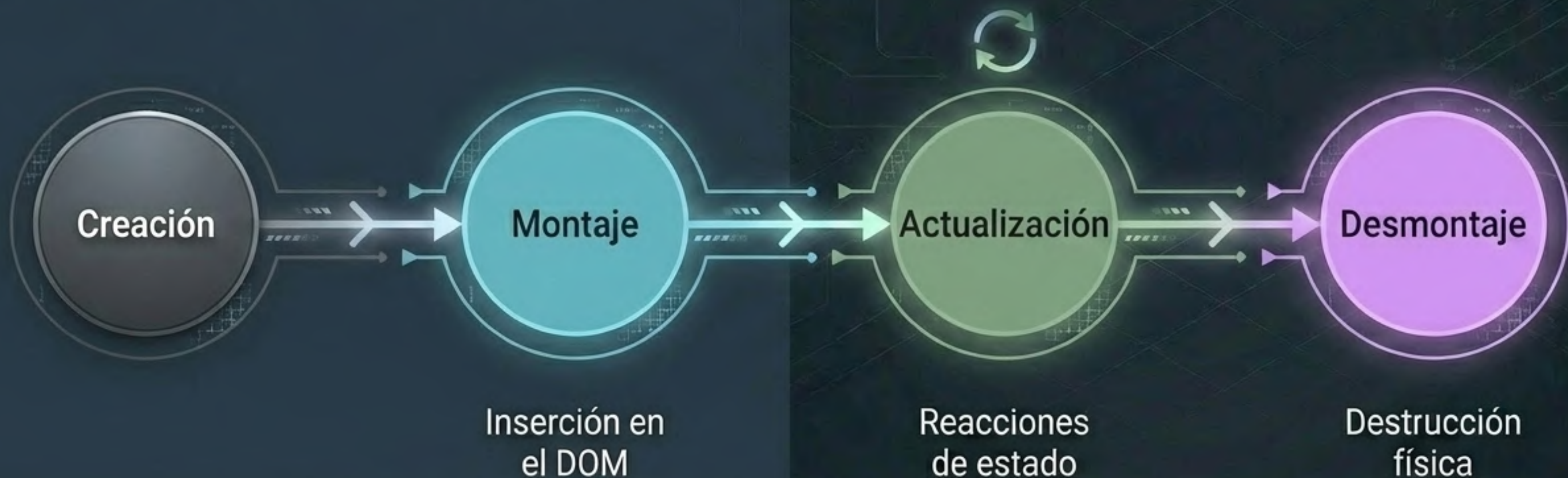
Síntesis: Props vs. Emits

	Props	Emits
Dirección	Padre → Hijo	Hijo → Padre
Propósito	Entregan datos de estado	Notifican acciones o sucesos
Naturaleza	Variables de solo lectura	Disparadores de eventos

Regla mental: El padre informa; el hijo avisa.

Ciclo de Vida: La línea del tiempo del componente

Los componentes no son estáticos. Nacen, reaccionan a los cambios de estado y son **destruidos** cuando la aplicación ya no los necesita.



En Composition API, la lógica inicial suele ejecutarse directamente en **<script setup>**; los hooks se usan para momentos específicos como montaje, actualización o desmontaje.

Hooks del Ciclo de Vida con Composition API

```
1 <script setup>
2 import { ref, onMounted, onUpdated, onBeforeUnmount } from 'vue'
3
4 const contador = ref(0)
5
6 onMounted(() => console.log('Montado en el DOM'))
7 onUpdated(() => console.log('El DOM reaccionó al contador'))
8 onBeforeUnmount(() => console.log('A punto de ser destruido'))
9 </script>
10
11 <template>
12   <button @click="contador++">Contador: {{ contador }}</button>
13 </template>
```

`onUpdated` puede ejecutarse muchas veces. Evita lógica pesada o cambios reactivos innecesarios dentro de este hook.



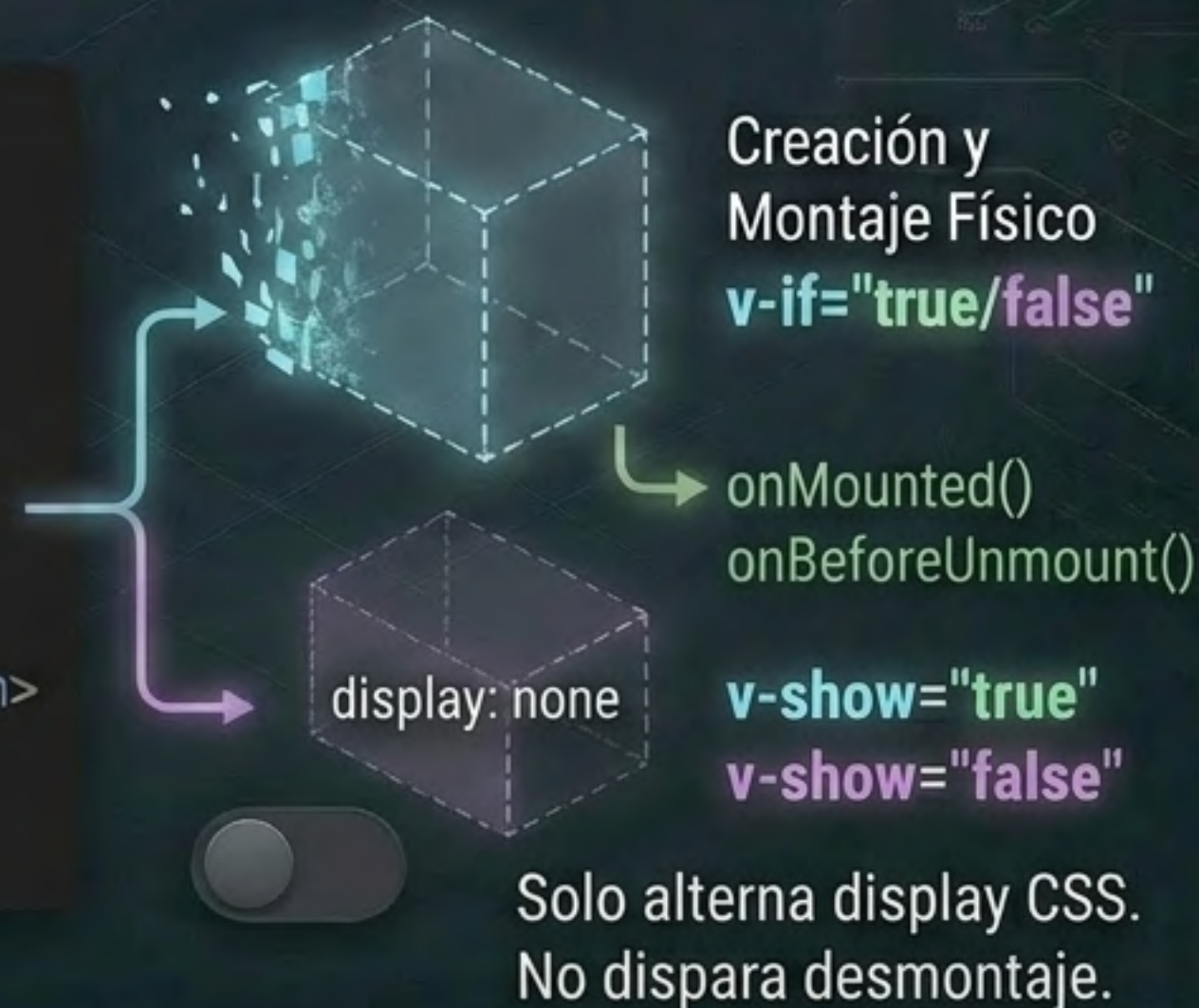
Montaje y desmontaje físico con v-if y v-show

v-if crea y destruye el componente.

v-show solo alterna display: none y no dispara desmontaje.



```
1 <script setup>
2 import { ref } from 'vue'
3 import Padre from './components/Padre.vue'
4
5 const mostrar = ref(true)
6 </script>
7
8 <template>
9   <button @click="mostrar = !mostrar">Alternar Existencia</button>
10   <Padre v-if="mostrar" />
11 </template>
```



Slots: Proyección de contenido flexible

- Espacios reservados donde el padre puede inyectar HTML arbitrario o estructuras completas, no solo variables.
- Props pasan datos. Slots pasan contenido o estructura visual.

CardBase.vue

```
1 <template>
2   <article class="card">
3     <slot />
4   </article>
5 </template>
```

Uso en el Padre

```
1 <CardBase>
2   <h3>Título inyectado</h3>
3   <p>Contenido flexible desde el padre.</p>
4 </CardBase>
```

<h3>Título inyectado</h3>

<p>Contenido flexible desde el padre.</p>

<slot />

Componentes Dinámicos y Arquitectura General

```
import { shallowRef } from 'vue'
import VistaUno from './VistaUno.vue'
import VistaDos from './VistaDos.vue'
```

Un componente dinámico permite alternar qué componente se muestra sin usar Vue Router. Usamos **shallowRef** porque estamos guardando componentes, no datos comunes como *strings* o números.



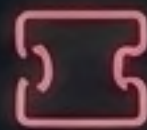
Props

→ bajan datos



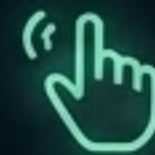
Emits

→ suben
eventos



Slots

→ insertan
contenido



Hooks

→ controlan
etapas



Dinámicos

→ alternan vistas

Si se necesita conservar el estado entre cambios, se puede usar **<KeepAlive>**.